

Ecco un'altra variante d'esame sullo stesso stile, ma incentrata su una tematica diversa (gestione di una **Biblioteca/Mediaforge**) e con un focus su un calcolo statistico differente (conteggio di elementi anziché media/somma), mantenendo intatti i vincoli di programmazione a oggetti e di efficienza sul BST.

Simulazione d'Esame: BST Risorse Multimediali

Considerando la seguente gerarchia di classi, definire un programma in C++ che implementa le funzionalità descritte di seguito.

a) Creazione degli oggetti

Creare 6 oggetti `Risorsa` : 2 Libri, 2 Nuovi Media (es. E-book/Audiolibri) e 2 Periodici (Riviste). Prevedere per ciascuna risorsa almeno gli attributi `titolo` , `codice_id` (numerico univoco) e `dimensione_pagine` (numero di pagine o equivalente cartaceo). I valori possono essere assegnati in modo arbitrario.

Per l'attributo `dimensione_pagine` considerare i seguenti intervalli indicativi:

- I **Periodici** hanno un numero di pagine compreso tra 20 e 120.
- I **Nuovi Media** hanno un equivalente di pagine compreso tra 100 e 500.
- I **Libri** hanno un numero di pagine compreso tra 200 e 1000.

b) Inserimento nel BST

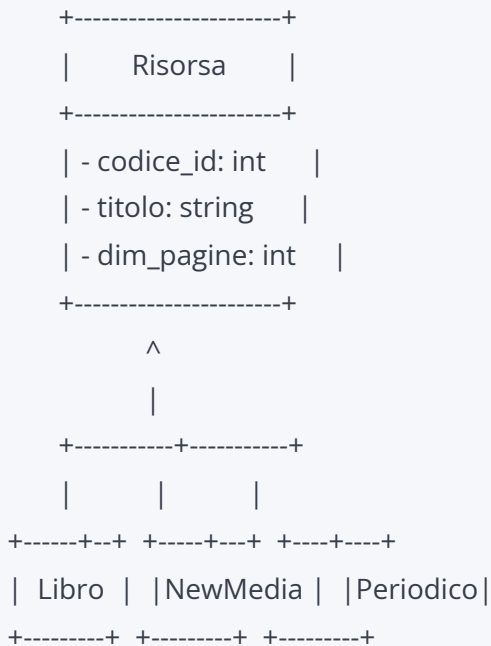
Inserire gli oggetti `Risorsa` creati in un unico Albero Binario di Ricerca (BST), considerando il campo `codice_id` come chiave di ordinamento (oppure, a scelta, la `dimensione_pagine`). L'albero deve ospitare puntatori alla classe base `Risorsa` per sfruttare il polimorfismo.

c) Conteggio con visita singola

Implementare una funzione C++ che prende in input la radice del BST e determina **quante risorse per ciascuna categoria** superano una determinata soglia di pagine (passata come parametro alla funzione).

- L'output dovrà mostrare il numero di Libri, di Nuovi Media e di Periodici che superano la soglia.
- **Vincolo:** Il calcolo va implementato effettuando **una sola visita** della struttura dati (evitando di scorrere l'albero tre volte).

Diagramma delle Classi UML



Requisiti di implementazione:

1. **Polimorfismo:** Utilizzare i metodi virtuali puri nella classe base se necessario, rendendola una classe astratta, o metodi virtuali standard per il recupero delle informazioni specifiche di tipo.
2. **Memory Management:** Assicurarsi che il distruttore della classe base sia `virtual` per garantire la corretta deallocazione della memoria dinamica dei nodi del BST.
3. **Main di test:** Includere una funzione `main()` che crei i nodi, li inserisca nel BST in modo non ordinato (per testare le proprietà dell'albero) e interroghi la funzione di conteggio con una soglia a scelta (es. 150 pagine).